

Mtech Software Engineering

Software Quality Assurance and Testing (S2-25_SEZG501)

Objective : The objective is to design a comprehensive management and quality governance framework for a large-scale, multi-year software project distributed across global locations.

It aims to demonstrate your ability to synchronize project scheduling, testing, version control, and auditing to ensure consistent quality in a complex, high-stakes engineering environment.

Group Members:

- ★ Sakthi P: 2025TM93220
- ★ Saif Uddin Ahamed: 2025TM93168
- ★ Sajja Chandana: 2025TM93117
- ★ Salandri Shirisha: 2025TM93131
- ★ Saloni Kashyap: 2025TM93083
- ★ Sanika Vineet kshirsagar: 2025TM93040
- ★ Sanjai Siddharthan M: 2025TM93014
- ★ Sanjana Nayak: 2025TM93056

Application Name: AI Scrum Master Assistant for JIRA

Concept: A cross-platform AI assistant integrated into JIRA that automates the "busy work" of a Scrum Master. It uses LLMs to analyze velocity, predict sprint slippage, and facilitate ceremonies for distributed teams across time zones.

1. Introduction

A cross-platform AI assistant integrated into JIRA that automates the "busy work" of a Scrum Master. It uses LLMs to analyze velocity, predict sprint slippage, and facilitate ceremonies for distributed teams across time zones.

The AI Scrum Master Assistant for JIRA is an intelligent tool designed to automate and enhance Agile project management processes within JIRA. It leverages AI/ML to assist Scrum Masters and teams in sprint planning, backlog prioritization, risk identification, and performance tracking. The goal is to improve team productivity, reduce manual effort, and enable data-driven decision-making in distributed Agile environments.

2. Product Requirements

2.1. Functional Requirements

- **Automated Sprint Planning:** The system shall analyze historical velocity, team capacity, and backlog complexity to suggest optimal task allocations for upcoming sprints.
- **Intelligent Backlog Prioritization:** The assistant must rank backlog items based on business value, technical dependencies, and historical data to assist Product Owners in decision-making.
- **Ceremony Automation:**
 - **Daily Standup Summaries:** The system shall automatically generate concise summaries from team updates to highlight progress and blockers.
 - **Retrospective Insights:** Future iterations must provide AI-driven suggestions for process improvements based on sprint performance.
- **Predictive Risk Management:** The assistant must identify potential delays, resource bottlenecks, or "sprint slippage" before they occur, providing actionable alerts to the Scrum Master.
- **Dynamic Reporting:** The system shall provide real-time analytics, including burndown charts and velocity insights, synchronized directly with JIRA APIs.

2.2. AI & Machine Learning Requirements

- **Model Accuracy & Reliability:** Predictions regarding sprint outcomes and risk must be validated against historical JIRA datasets to ensure high precision and recall.
- **Explainability:** AI-generated recommendations (such as why a task was prioritized) must be logical and transparent to ensure they are trustworthy for human stakeholders.
- **Continuous Learning:** The system should have the capability to retrain or update its logic

over time as it processes more team-specific data.

- **Bias Mitigation:** The AI must be tested to ensure it provides neutral recommendations that do not unfairly prioritize specific team members or task types.

2.3. Non-Functional & Technical Requirements

- **Scalability:** The architecture must support concurrent usage by 5–6 distributed teams (approximately 50–80 users) without performance degradation.
- **Security & Data Privacy:**
 - **Encryption:** All project data transmitted between JIRA and the AI assistant must be encrypted.
 - **Access Control:** The system must enforce Role-Based Access Control (RBAC) consistent with JIRA permissions (Scrum Master, Developer, Product Owner).
- **Integration Fidelity:** The system must maintain real-time data synchronization with JIRA via webhooks and APIs, ensuring that status changes in JIRA are reflected instantly in the assistant.
- **Availability:** The assistant must be accessible across different geographies and time zones, utilizing cloud-based environments to ensure 24/7 availability for distributed teams.

2.4. Governance & Quality Requirements

- **Traceability:** All requirements must be traceable from the initial SRS through to the test cases and final deployment baselines.
- **Maintainability:** The code and AI logic must adhere to enterprise coding standards and undergo mandatory peer reviews via GitHub Pull Requests to ensure readability.
- **Compliance:** The project must undergo quarterly process and security audits to ensure adherence to Agile practices and data privacy standards.

3. Project Schedule and Plan

Project: AI Scrum Master Assistant for Jira (Duration: 30 Months)

3.1. Master Project Schedule

Phase	Sub-Activities	Duration (Months)	Timeline	Owner	Deliverables	Dependencies
Initiation & Feasibility	Business case, stakeholder identification, feasibility study	2 Months	Month 1-2	Program Manager	Project Charter, Feasibility Report	None
Requirement	Requirement gathering,	4 Months	Month 3-6	Product Owner	SRS, Product	Stakeholder inputs

Engineering	analysis, backlog creation, prioritization				Backlog	
Architecture & Design	HLD, LLD, architecture validation, design reviews	4 Months	Month 7-10	Architect	Design Documents	Approved requirements
Development (Incremental)	Feature development, unit testing, code reviews	12 Months	Month 11-22	Dev Leads	Working modules	Design completion
System & Integration Testing	Integration testing, system validation, defect fixing	6 Months (Overlap)	Month 17-22	QA Leads	Test Reports, Defect Logs	Development progress
UAT & Pre-Production	User validation, feedback incorporation	1 Month	Month 23	Business + QA	UAT Sign-off	Stable build
Deployment & Rollout	Production deployment, release management	1 Month	Month 24	DevOps	Live system	UAT approval
Hypercare & Enhancements	Monitoring, bug fixes, minor enhancements	6 Months	Month 25-30	Support Team	Stabilized system	Production release

3.2. Program Increment (PI) Plan

PI Cycle	Duration	Activities	Owner	Deliverables
PI Planning	2 Days per Quarter	Planning, backlog prioritization	Program Manager, Product Owner	PI Objectives
Execution (5 Sprints)	10 Weeks	Development, testing,	Dev & QA Teams	Incremental features

		integration		
System Demo	End of PI	Feature demonstration	All Teams	Demo report
Inspect & Adapt	2 Days	Performance review, improvements	QA Manager	Improvement plan

3.3. Sprint-Level Schedule (2 Weeks per Sprint)

Day/Activity	Description	Owner	Output
Day 1	Sprint Planning	Scrum Master, Team	Sprint backlog
Day 2-10	Development & Unit Testing	Developers	Code commits
Day 3-12	Functional & API Testing	QA Team	Test results
Daily	Daily Standup	All	Status updates
Day 11-12	Regression Testing	QA Team	Regression report
Day 13	Sprint Review	Stakeholders	Demo
Day 14	Retrospective	Team	Action items

3.4. Work Breakdown Structure (WBS)

Module	Activities	Owner	Timeline
Jira Integration	API integration, webhook setup	Backend Team	Month 11-14
AI/NLP Engine	Model development, training, validation	AI Team	Month 12-18
Dashboard & Analytics	UI development, reporting	Frontend Team	Month 13-19
Risk Prediction	Algorithm development, testing	AI Team	Month 15-20
Chat Assistant	Integration with communication tools	Dev Team	Month 18-21
Security Module	Authentication, authorization	DevOps/Security	Month 14-20

3.5. QA & Testing Schedule

Testing Phase	Activities	Duration	Timeline	Owner	Deliverables
Test Planning	Strategy, RTM, test design	2 Months	Month 5-6	SQA Manager	Test Plan
Test Design	Test cases, automation scripts	3 Months	Month 7-9	QA Team	Test Cases
Continuous Testing	Functional, API testing	12 Months	Month 11-22	QA Team	Execution reports
AI Testing	Model accuracy, bias testing	6 Months	Month 15-20	QA + AI Team	Validation reports
Performance Testing	Load, stress testing	2 Months	Month 20-21	QA Team	Performance report
Security Testing	Vulnerability testing	2 Months	Month 20-21	Security Team	Security report
UAT	Business	1 Month	Month 23	QA Team	UAT

Support	validation				sign-off
---------	------------	--	--	--	----------

3.6. Milestone Plan

Milestone	Timeline	Owner	Deliverable
Project Kickoff	Month 1	Program Manager	Kickoff report
Requirement Freeze	Month 6	Product Owner	Approved SRS
Design Sign-off	Month 10	Architect	Final Design
MVP Release	Month 16	Dev Team	MVP Build
Beta Release	Month 20	QA Team	Beta Version
Final Release	Month 24	DevOps	Production System
Project Closure	Month 30	Program Manager	Closure Report

3.7. Resource Allocation Plan

Role	Count	Responsibility	Engagement Duration
Program Manager	1	Overall delivery	Full project
Product Owner	1	Requirements	Full project
Architects	2	Design	Month 3-10
Developers	20-25	Development	Month 11-22
QA Engineers	10-12	Testing	Month 7-23
AI/ML Engineers	5-6	AI modules	Month 12-20
DevOps Engineers	2-3	CI/CD, deployment	Month 10-24
Support Team	5	Maintenance	Month 25-30

3.8. Dependency & Risk Schedule

Dependency	Impact	Phase Affected	Mitigation
------------	--------	----------------	------------

Jira API availability	High	Development	Early integration
AI model readiness	High	Testing	Parallel development
Cross-team coordination	High	All phases	Scrum of Scrums
Infrastructure setup	Medium	Development	Early provisioning

3.9. Quality Gate Schedule

Phase	Quality Gate	Criteria	Owner
Requirements	Sign-off	Approved SRS	Product Owner
Design	Review Approval	Design validation	Architect
Development	Sprint Exit	No critical defects	QA Lead
Testing	Test Completion	$\geq 85\%$ coverage	SQA Manager
Release	Go-Live Approval	Zero critical defects	QA + Business

4. Test Planning

4.1 Overview of Testing Objectives & Testing Scope

Purpose of Testing

The primary purpose of testing this system is to ensure that the product is reliable, scalable, secure, and intelligent, especially in a large-scale, distributed Agile environment.

Testing Objectives

1. Functional Correctness Validation

The system must correctly implement all intended features, including sprint planning, reporting, and AI recommendations.

- Validate workflows such as sprint creation, backlog updates, and report generation
- Ensure user actions produce expected outputs
- Verify role-based access (Scrum Master, Developer, Product Owner)

2. Integration Testing with Jira APIs

The assistant depends on deep integration with Jira, making API validation critical.

- Validate data synchronization between Jira and the AI system
- Ensure compatibility with different Jira configurations
- Test API failure handling and fallback mechanisms
- Verify real-time updates (e.g., issue status changes reflected instantly)

3. Performance and Scalability Assurance

Given the project scale (5–6 teams, multiple locations), the system must perform efficiently under load.

- Test concurrent usage by multiple teams
- Measure response time for AI recommendations
- Validate system stability during peak sprint activities
- Ensure scalability for long-term usage (2–3 years of data growth)

4. Security and Compliance Validation

The system handles sensitive project and organizational data.

- Ensure secure data transmission (encryption)
- Validate authentication and authorization mechanisms
- Test for vulnerabilities (SQL injection, API abuse, etc.)
- Ensure compliance with enterprise security standard

5. User Experience and Usability Testing

The product must be intuitive and easy to use for Agile teams.

- Validate UI clarity and ease of navigation
- Ensure AI suggestions are understandable and actionable
- Test accessibility for distributed teams with varying expertise
- Gather feedback during UAT for continuous improvement

4.2 Testing Strategy & Testing Types

The project follows a structured **multi-level testing strategy**, where each level focuses on validating different aspects of the system.

Testing Level	What We Test in Our Product	How We Execute (Project-Specific Approach)	Owner
Unit Testing	Individual modules such as AI prediction functions, Jira API handlers, data processing logic	Developers write unit tests for AI algorithms (e.g., sprint prediction logic), mock Jira APIs, and validate edge cases like missing data or incorrect inputs. Continuous execution using CI pipelines ensures early	Developers

Integration Testing	Interaction between AI engine, Jira APIs, databases, and reporting modules	Validate data flow from Jira → AI engine → output dashboards. Test API failures, latency issues, and synchronization across distributed environments. Use real Jira sandbox environments.	QA Team
System Testing	Complete end-to-end workflow of the product	Simulate real Agile workflows: sprint planning → execution → reporting → retrospectives. Validate how AI suggestions behave in real scenarios with multiple teams and large datasets	QA Team
User Acceptance Testing (UAT)	Real-world usability and business validation	Scrum Masters and Product Owners test features like backlog prioritization and risk alerts.	Product Owner / Stakeholders

Regression Testing	Existing features after new updates	Automated test suites run after every sprint to ensure new features (e.g., improved AI model) do not break existing functionalities like reporting or API integration	QA Automation Team
--------------------	-------------------------------------	---	--------------------

- **Testing Types**

A. Functional Testing

This ensures that all features work according to requirements and produce correct outputs.

Feature	What we Test	How We Test in Our Project
Sprint Planning Recommendations	Accuracy of sprint suggestions, workload distribution, velocity calculation	Provide historical sprint data and verify AI-generated plans. Compare AI recommendations with manual planning by Scrum Masters. Test edge cases like new teams (no history), sudden backlog spikes, and incomplete da

Risk Prediction Engine	Detection of delays, blockers, and resource risks	Simulate scenarios like delayed tasks, overloaded team members, and dependency conflicts. Validate whether AI correctly predicts risks and provides actionable alerts. Measure false positives and false negatives.
Automated Reporting	Correctness of generated reports and insights	Validate report data against Jira source data. Check real-time updates, formatting, and completeness. Ensure reports reflect accurate sprint metrics (velocity, burndown, completion rate)

B. Non-Functional Testing

This ensures the system performs well under real-world conditions.

Testing Type	What We Test	How We Execute for Our Product
Performance Testing	System response time, scalability with multiple teams	Simulate 5–6 teams (50–80 users) working simultaneously. Use load testing tools to measure response time of AI recommendations and report generation. Validate system stability

		during sprint deadlines (peak load).
Security Testing	Data protection, API security, user authentication	Test encryption of Jira data, validate secure API communication, and perform vulnerability testing (e.g., unauthorized access attempts). Ensure role-based access control (RBAC) is properly enforced.
Usability Testing	Ease of use, UI clarity, user interaction	Conduct testing sessions with Scrum Masters and developers. Evaluate whether AI suggestions are easy to understand, dashboards are intuitive, and workflows are smooth for distributed teams.

C. AI-Specific Testing

Since the system is AI-driven, specialized testing is required beyond traditional methods.

AI Testing Type	What We Test	How We Execute (Project-Specific Approach)
Model Accuracy Validation	Correctness of AI predictions and recommendations	Use historical Jira project data to compare predicted vs actual outcomes. Measure accuracy

		metrics (precision, recall). Continuously retrain and validate models using updated datasets.
Bias Detection	Fairness and neutrality of AI decisions	Analyze whether AI unfairly prioritizes certain tasks, team members, or patterns. Test with diverse datasets to ensure unbiased recommendations across teams and projects.

4.3 Test Deliverables

The testing process will produce structured deliverables to ensure transparency and traceability throughout the project.

The key deliverable is the **Test Plan Document**, which defines the testing scope, strategy, and tools for the AI-integrated Jira system. **Test cases and scenarios** will cover all major features such as sprint planning, risk prediction, and reporting.

Automation scripts will be developed for UI, API, and regression testing and maintained across sprints. All defects will be logged and tracked in Jira with proper severity and status.

Test data sets, including synthetic and anonymized Jira data, will be used for validating AI models. At the end of each cycle, a **Test Summary Report** will provide insights into execution status, defects, AI accuracy, and overall quality.

5. Configuration Management Plan

5.1. Objective

The objective of the Configuration Management (CM) plan is to establish a systematic framework to identify, control, and audit all artifacts generated throughout the 30-month lifecycle of the AI Scrum master project.

5.2. Configuration Identification (Configuration Items - CIs)

To maintain control over the complex architecture, Configuration Items (CIs) are categorized into three distinct streams:

Software & Application CIs	AI/ML Specific CIs	Documentation & Governance CIs
● Source code ● UI Components ● JIRA API webhook integrations ● Middleware logic ● Database schemas ● Release packages	● Training datasets (anonymized JIRA data used for model development) ● LLM prompts ● Model weights (e.g. Tensorflow artifacts) ● Hyperparameters ● Data pre-processing scripts	● Software Requirement Specification (SRS) ● High-Level/Low-Level Design (HLD/LLD) documents ● Test Plans ● Test Cases ● Test Datasets ● Test Scripts (Selenium, Postman) ● Release Notes

5.3. Version Control & Branching Strategy

To support concurrent development by 20-25 developers and 5-6 AI engineers, the project will adopt a strict GitFlow branching strategy.

Main Branch	Contains the stable, production-ready code. Commits here represent official releases.
Develop Branch	The integration branch for upcoming Program Increment (PI) features.
Feature Branch	Created from develop for specific user stories (e.g., "Automated Sprint Planning

	logic").
Release Branch	Branched from develop branch during the "UAT & Pre-Production" phase for final bug fixes before merging to main branch .

5.4. Configuration Control (Change Management)

Modifications to baselined configuration items must follow a governed process to prevent unauthorized or untested changes:

1. **Pull Requests (PR):** All code and test script changes must be submitted via GitHub PRs.
2. **Automated Checks:** GitHub Actions will automatically trigger Unit Tests, SonarQube static analysis, and OWASP ZAP security scans upon PR creation.
3. **Peer Review:** A mandatory asynchronous review by at least one peer and one Architect/Lead is required before merging into the develop branch.
4. **Configuration Control Board (CCB):** Major architectural changes or modifications to the core AI prediction logic require approval from the CCB (comprising the Program Manager, Product Owner, and Lead Architect).

5.5. AI/ML Configuration Management (MLOps)

Unlike traditional code, AI models require simultaneous versioning of code, data, and models.

- **Experiment Tracking:** MLflow will be utilized to track all AI model experiments, recording hyperparameters, precision/recall metrics, and the specific dataset version used for training.
- **Data Versioning:** Datasets used for training the predictive risk management and backlog prioritization models will be versioned to ensure AI explainability and reproducibility. If an AI recommendation is flagged for bias, the exact training data and model state can be audited.

5.6. Baseline Management

Baselines act as stable reference points for the project and will be established at key milestones defined in the Master Project Schedule:

Requirement Baseline: Established at Month 6 upon SRS sign-off.

Sprint Baseline: Codebase tagged at the end of every 2-week sprint following successful regression testing.

Release Baseline: Immutable tags created in GitHub for the MVP Release (Month 16), Beta Release (Month 20), and Final Production Release (Month 24).

5.7. Configuration Audits

Functional Configuration Audit (FCA): Conducted prior to the MVP and Beta releases to ensure the software and AI modules perform exactly as specified in the SRS.

Physical Configuration Audit (PCA): Conducted by the Configuration Manager to verify that the release packages in Docker/Kubernetes exactly match the documented, baselined versions in GitHub.

5.8. Configuration Management Tools & Infrastructure

Purpose	Selected Tool	Reason for Selection
Version Control	GitHub Enterprise	Industry standard for managing source code and documentation; supports a seamless Pull Request (PR) process for asynchronous peer reviews across multiple geo-locations.
CI/CD	GitHub Actions	Provides native integration with the version control system, allowing for automated testing and deployment triggers with every code commit.
Containerization	Docker / Kubernetes	Ensures environment consistency across development, testing, and production; essential for managing microservices and AI model dependencies.
Static Code Analysis	SonarQube	Automates the detection of bugs, vulnerabilities, and "code smells" to maintain high enterprise coding standards.
AI Model Lifecycle	MLflow	Critical for tracking AI

		experiments, versioning model weights, and ensuring reproducibility of LLM-driven predictions.
Audit & Governance	ServiceNow	Provides a robust, automated workflow for tracking formal audits and managing change requests for baselined items.

6. Review and Audit Plan

6.1. Purpose

To define a systematic and formal approach to evaluate the quality, compliance, and correctness of all key project artifacts and processes. This is essential for a long-running, distributed project to prevent quality degradation, ensure process adherence, and enable early detection of risks.

6.2. Scope

- Requirements and user stories
- Architecture and design documents
- Source code and AI models
- Test artifacts and test results
- Configuration items and baselines
- Release readiness and production deployments
- Process compliance with Agile, security, and quality standards

6.3. Ownership and Accountability

Review Ownership

Artifact	Review Owner
Requirements Documents	Product Manager
Architecture & Design	Solution Architect
Source Code	Development Leads
Test Strategy & Test Cases	Test Manager
Automation Framework	SDET Lead

Audit Ownership

Audit Type	Owner
Process Audit	Internal Quality Team
Configuration Audit	Configuration Manager
Release Audit	Internal Audit + Stakeholders
Compliance & Security Audit	External / Specialized Team

6.4. Review Plan (Static Quality Assurance)

- **Requirements Review:** To ensure business needs are correctly captured and requirements are clear, testable, and traceable.
- **Architecture and Design Review:** To focus on scalability, data pipelines, security, and integration.
- **Code Review:** To ensure adherence to coding standards, quality of AI logic, and security.
- **Test Artifact Review:** To ensure test cases align with requirements and AI scenarios are validated.

6.5. Audit Plan (Formal Compliance Verification)

- **Process Audit:** To verify adherence to defined processes such as Agile practices, defect management, and change management.
- **Configuration Audit:** To ensure all configuration items are correctly identified, versioned, and baselined.
- **Release Audit:** To ensure the product is ready for production deployment.

6.6. Metrics for Effectiveness

- Defects detected per review
- Percentage rework due to audit findings
- Audit closure time
- Recurring non-conformances
- Review coverage %

7. Tools Comparison and Selection:

Category	Tool	Type	Primary Use in Our Project	Key Features	Final Decision & Justification
Test Management	Zephyr	Paid	Manage test cases, traceability with Jira issues	Native Jira integration, test tracking, reporting dashboards	Selected – Best fit since our product is built on Jira ecosystem
Test Management	TestRail	Paid	Advanced test case management & reporting	Rich reporting, customizable workflows	Not selected – Integration overhead
Test	TestLink		Basic test	Free,	Backup

Management		Open Source	case management	simple test tracking	option only
UI Automation	Selenium	Open Source	Automate UI testing of dashboards & reports	Cross-browser support, flexible scripting	Selected – Suitable for complex UI workflows
UI Automation	Cypress	Open Source	Fast UI testing for frontend components	Real-time reload, fast execution	Consider – For faster test cycles

API Testing	Postman	Freemium	Test Jira API integratio ns	Easy API testing, automation support	Selected – Ideal for API validation
Performanc e Testing	JMeter	Open Source	Load testing with multiple teams/users	Supports distributed load testing	Selected – Best for large-scale simulation
Performanc e Testing	LoadRunne r	Paid	Enterprise-l evel performanc e testing	Advanced analytics, enterprise support	Not selected – Cost constraint

AI/ML Tools	TensorFlow	Open Source	Build & validate AI models	Model training, prediction pipelines	Selected – Core AI engine
AI/ML Tools	MLflow	Open Source	Track AI experiment s & model versions	Model versioning, experiment tracking	Selected – Critical for AI lifecycle
CI/CD	Jenkins	Open Source	Automate build & test pipelines	Continuous integration, scheduling	Selected – Strong CI/CD support
Version Control	GitHub	Freemium	Store code & test scripts	Version control, collaboratio n tools	Selected – Industry standard

8. Team, Ownership, and Meetings

8.1. Team Members and Division of Work

Team members	Division of work
Salandri Shirisha :2025TM93131	Introduction,Test Planning and brainstorming discussion inputs
Sakthi P: 2025TM93220	Project Schedule, Meeting,inputs flow
Sanjana Nayak: 2025TM93056	Configuration Management plan,Input flow
Sajja Chandana: 2025TM93117	Review and Audit plan ,final edits,split the work
Saif Uddin Ahamed: 2025TM93168	Testing strategy and Testing types followed by Configuration Management plan and reviewing the document structure.
Sanika Vineet kshirsagar: 2025TM93040	Project Schedule and plan,Group discussion,
Sanjai Siddharthan M: 2025TM93014	Product Requirements, AI features, Ideation/Brainstorming, and documentation structure.
Saloni Kashyap: 2025TM93083	Tools comparison,flow, meeting schedule

8.2. Communication & Meeting Schedule

Meeting	Frequency	Participants	Objective
Daily Standup	Daily	Team	Progress tracking
Scrum of Scrums	3/week	Scrum Masters	Dependency resolution
Sprint Review	Bi-weekly	Stakeholders	Demo
Retrospective	Bi-weekly	Team	Improvement
PI Planning	Quarterly	All Teams	Alignment
QA Sync	Weekly	QA Teams	Quality tracking
Requirements Review Meeting	As required	PM, BA, QA, Architect	
Design Review Board	Quarterly	Architects, Leads	

Audit Kick-off	Quarterly	Audit Team, Managers	
Audit Closure Meeting	Quarterly	Quality Manager	

8.3. Meeting Records

8.3.1 Meeting Record 1

Date: 12-Apr-2026

Time: 10:00-11:30 IST

Attendees: Salandri Shirisha, Sanjai Siddharthan, Sajja Chandana

Key Decisions:

- Features and Requirement of the Project
- Mandatory review of AI explainability features
- Introduced security audit for ML APIs
- Separate audit checklist for data privacy compliance

8.3.2 Meeting Record 2

Date: 18-Apr-2026

Time: 3:00–4:30 IST

Attendees: Sakthi P, Sanika Vineet, Sanjana Nayak, Saloni Kashyap

Key Decisions:

- Finalized tool selection strategy (Jira, GitHub, SonarQube, MLflow)
- Agreed to use GitFlow branching strategy for version control
- Decided to maintain traceability matrix (RTM) linking requirements → test cases
- Identified need for automated CI/CD pipeline using GitHub Actions

8.3.3 Meeting Record 3

Date: 25-Apr-2026

Time: 11:00–12:30 IST

Attendees: Saif Uddin Ahamed, Sajja Chandana, Sanjai Siddharthan, Salandri Shirisha

Key Decisions:

- Defined **AI testing strategy** including accuracy, bias, and explainability validation
- Planned use of **historical Jira datasets** for model training and validation
- Introduced **AI-specific test metrics** (precision, recall, false positives)
- Agreed on **continuous retraining approach** for improving AI models

8.3.4 Meeting Record 4

Date: 30-Apr-2026

Time: 5:00–6:00 IST

Attendees: Entire Team

Key Decisions:

- Finalized **Master Project Schedule (30 months)**
- Confirmed **Program Increment (PI) planning structure**
- Decided on **2-week sprint cycles** with defined ceremonies
- Allocated responsibilities for **WBS modules (AI, UI, Integration, Security)**

Thank you!